Práctica 3

Uso de interrupciones para leer valores de un botón

Grado en Ingeniería en Robótica Software
Universidad Rey Juan Carlos



©2024 Julio Vega Pérez
©2026 Esther Aguado
Algunos derechos reservados.
Este trabajo se entrega bajo licencia CC-BY-SA 4.0.

1. Introducción

Las interrupciones son un mecanismo ampliamente utilizado en la programación embebida, ya que permiten que el microcontrolador responda a eventos externos en cualquier momento, incluso mientras se ejecuta el programa principal. Este enfoque resulta especialmente útil en situaciones donde se requiere una reacción inmediata, como al leer un encoder para no saltarnos ningún pulso o recibir datos a través del puerto serie por parte del usuario.

En esta práctica vamos a ver cómo manejar eventos asíncronos en Arduino mediante el uso de interrupciones. Para ello, utilizaremos un pulsador como dispositivo de entrada, y nuestro objetivo será detectar cuándo se ha presionado el botón. Además, abordaremos el problema del rebote del botón y cómo mitigarlo mediante técnicas de antirrebote tanto por software como por hardware.

2. Configuración de pines para el manejo de eventos

Hasta ahora hemos trabajado con los pines en su configuración de salida (*OUTPUT*), para manejar convenientemente un LED. En esta configuración, somos nosotros quienes establecemos el nivel lógico de la señal de salida. Sin embargo, cuando configuramos un pin como entrada (*INPUT*), debemos asegurarnos de que la tensión aplicada esté definida.

Si el pin no está conectado a una señal ni dispone de un circuito que establezca su nivel lógico, queda en estado flotante. En estas condiciones, puede interpretar como HIGH o LOW las perturbaciones eléctricas del entorno, por lo que sus lecturas no son fiables. Para evitarlo, utilizamos resistencias *pull-up* o *pull-down*, que mantienen la entrada en un nivel lógico definido mientras no actúa la señal externa.

Una resistencia *pull-up* se conecta entre el pin de entrada y la alimentación positiva de la placa (5 V). Así, cuando no actúa ninguna señal externa, la entrada permanece en nivel alto (HIGH). Si un pulsador conecta el pin a tierra (GND), la entrada pasa a nivel bajo (LOW).

Por el contrario, una resistencia *pull-down* se conecta entre el pin de entrada y tierra (GND), manteniéndolo en nivel bajo (LOW) cuando no actúa ninguna señal externa. Al accionar un pulsador que conecta el pin a 5 V, la entrada pasa a nivel alto (HIGH). Los valores habituales de estas resistencias se sitúan entre 1 kΩ y 50 kΩ, siendo 10 kΩ la opción más habitual, ya que ofrece un equilibrio adecuado entre consumo de corriente y tolerancia al ruido.

3. Ejemplo de manejo de eventos

A continuación veremos diferentes ejemplos prácticos sobre cómo manejar eventos. Para estudiar de forma explícita el funcionamiento del circuito, utilizaremos una resistencia *pull-up* externa de 10 kΩ. La resistencia se conectará entre el pin de entrada y la alimentación de 5 V, de manera que el pin permanezca en nivel alto (HIGH) cuando el pulsador esté liberado. Al presionarlo, el pulsador conectará el pin a tierra (GND), haciendo que su lectura pase a nivel bajo (LOW).

El Arduino también dispone de resistencias *pull-up* internas, que pueden activarse mediante software con `pinMode(pin, INPUT_PULLUP)`. En esta práctica utilizaremos una resistencia externa de 10 kΩ, por lo que configuraremos el pin como `INPUT`.

Si se activa también la resistencia interna, ambas resistencias estarán conectadas en paralelo entre el pin de entrada y la alimentación. En consecuencia, la resistencia equivalente será menor que cualquiera de las dos resistencias por separado. Para estos ejemplos, utilizaremos únicamente la resistencia externa.

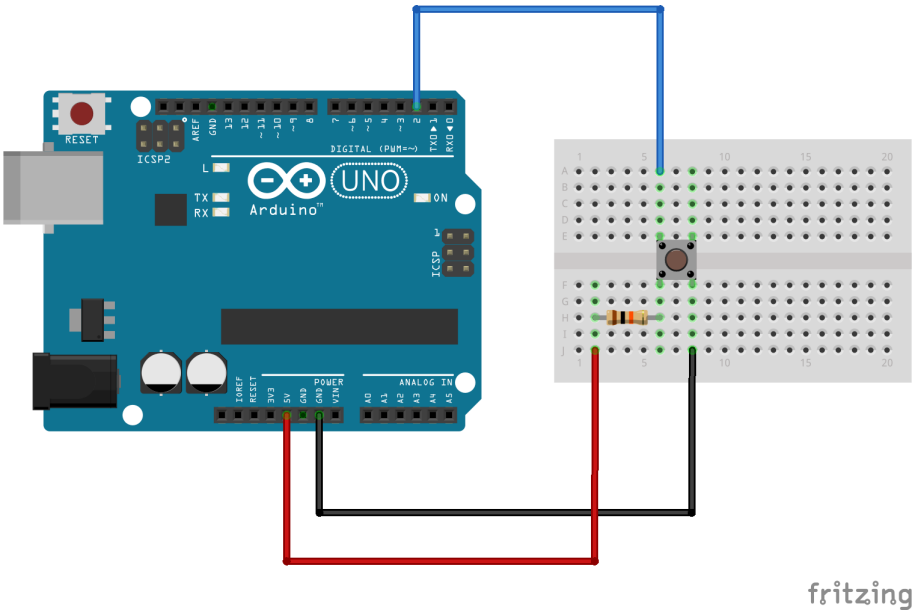


Figura 1: Esquema de conexión de pulsador empleado para estos ejemplos.

3.1. Lectura simple (*polling*)

La forma más directa de detectar una pulsación es consultar el estado del pin de forma periódica dentro de `loop()` mediante `digitalRead`. Esta técnica se conoce como *polling* (sondeo). Para contar pulsaciones no basta con comprobar si el pin está en LOW, ya que mientras el botón permanezca pulsado se contaría una y otra vez; hay que detectar el *cambio* de HIGH a LOW comparando la lectura actual con la anterior.

El siguiente ejemplo utiliza el archivo `boton_simple.ino`, con el mismo montaje de la Figura 1. Dentro de `loop()` se incluye un `delay(retardoTarea)` que simula otra tarea larga del programa (por ejemplo, leer sensores o mover motores).

```
1  const byte botonPin = 2;
2  const unsigned long retardoTarea = 1000;  // simula tarea larga
3
4  unsigned int contadorPulsaciones = 0;
```

```
int estadoAnterior = HIGH;

void setup() {
  Serial.begin(9600);
  pinMode(botonPin, INPUT);  // pull-up externa
}

void loop() {
  int estado = digitalRead(botonPin);

  // Flanco de bajada: el pin pasa de HIGH a LOW
  if (estadoAnterior == HIGH && estado == LOW) {
    contadorPulsaciones++;
    Serial.print("Pulsaciones detectadas: ");
    Serial.println(contadorPulsaciones);
  }
  estadoAnterior = estado;

  delay(retardoTarea);
}
```

Con `retardoTarea = 0` el programa consulta el pin continuamente y detecta las pulsaciones, aunque también los rebotes del pulsador (se estudian más adelante). Sin embargo, con `retardoTarea = 1000` el pin solo se lee una vez por segundo: una pulsación breve que comience y termine entre dos lecturas **no se detecta**. Este es el principal inconveniente del *polling*: la respuesta depende de lo que tarde el programa en volver a consultar el pin. Las interrupciones resuelven este problema.

3.2. Interrupciones con `attachInterrupt`

Una vez tenemos el funcionamiento más básico, podemos mejorar la eficiencia del programa utilizando interrupciones para asegurarnos que los eventos sean atendidos. En condiciones normales, cuando el programa comienza su ejecución, sigue estrictamente las instrucciones para las que ha sido diseñado. Sin embargo, esta secuencia ordenada se ve alterada cuando incorporamos el manejo de eventos. La principal particularidad de los eventos es que ocurren de forma asíncrona; es decir, mientras el programa continúa realizando sus propias tareas, puede producirse un evento que debe ser atendido de inmediato.

Para facilitar la gestión de eventos asíncronos muchos microcontroladores incluyen soporte para interrupciones. Cuando ocurre un evento externo, como pulsar un botón o recibir una señal, el flujo normal del programa puede interrumpirse en cualquier momento para ejecutar una función especial llamada rutina de servicio de interrupción (ISR, por sus siglas en inglés). Esta ISR funciona como un *callback*: se ejecuta de inmediato cuando se activa la interrupción, sin que el programa principal tenga que comprobar constantemente si el evento ha ocurrido.

Para configurar una interrupción en Arduino[1], se utiliza la función `attachInterrupt` dentro de la función `setup()`. La sintaxis básica es la siguiente:

```
void setup() {
  // otras inicializaciones
  attachInterrupt(digitalPinToInterrupt(pin), ISR, mode);
}
```

Esta función utiliza tres parámetros:

- `digitalPinToInterrupt(pin)`: indica el pin que generará la interrupción. No todos los pines de todas las placas ofrecen necesariamente las mismas posibilidades de interrupción; en el Arduino R4 estos pines son el 2 y 3.

[1] `https://docs.arduino.cc/language-reference/en/functions/external-interrupts/attachInterrupt/`

- `ISR`: nombre de la función que se ejecutará al atenderse la interrupción. La función ISR no debe devolver valor alguno ni aceptar parámetros.
- `mode`: define el evento que activará la interrupción. Los modos habituales son:
  - `RISING`: se activa ante un flanco de subida, cuando la señal cambia de LOW a HIGH.
  - `FALLING`: se activa ante un flanco de bajada, cuando la señal cambia de HIGH a LOW.
  - `CHANGE`: se activa cuando la señal cambia de estado, tanto de LOW a HIGH como de HIGH a LOW.
  - `LOW`: se activa mientras el pin permanece en nivel bajo, en las placas que admiten este modo.

El siguiente ejemplo utiliza el archivo `boton_interrupcion.ino`. El pulsador está conectado al pin digital 2 (D2), con una resistencia *pull-up* externa de 10 kΩ. Cuando se pulsa el botón, la entrada cambia de HIGH a LOW; por eso configuramos la interrupción con el modo `FALLING`. Con una resistencia *pull-down* y un pulsador que conectase la entrada a 5 V al presionarlo, la pulsación produciría un flanco de subida y podríamos utilizar `RISING`.

En esta primera versión contaremos las interrupciones detectadas, sin aplicar todavía antirrebote. De esta forma, podremos observar que una única pulsación física puede generar varios flancos de bajada debido al rebote mecánico. La rutina de interrupción únicamente incrementa un contador; la impresión por el puerto serie se realiza en `loop()`, no dentro de la ISR. Para poder comparar con el ejemplo anterior, `loop()` incluye la misma espera `retardoTarea`: aunque el programa principal esté ocupado, ninguna pulsación se pierde, ya que la ISR se ejecuta en el momento en que se produce el flanco.

```
const byte botonPin = 2;
const unsigned long retardoTarea = 1000;  // simula tarea larga
volatile unsigned int contadorInterrupciones = 0;
unsigned long ultimaLectura = 0;

void setup() {
  Serial.begin(9600);
  pinMode(botonPin, INPUT);  // resistencia pull-up externa

  attachInterrupt(digitalPinToInterrupt(botonPin), interrupcionBoton, FALLING);
}

void loop() {
  // Copiamos el contador con las interrupciones deshabilitadas
  // para evitar leerlo mientras la ISR lo está modificando.
  noInterrupts();
  unsigned long lectura = contadorInterrupciones;
  interrupts();

  // Mostramos el valor solo cuando haya cambiado.
  if (lectura != ultimaLectura) {
    Serial.print("Interrupciones detectadas: ");
    Serial.println(lectura);
    ultimaLectura = lectura;
  }

  delay(retardoTarea);
}

// Rutina de servicio de interrupción (ISR)
void interrupcionBoton() {
  contadorInterrupciones++;
}
```

Este ejemplo permite observar las interrupciones que se detectan, pero no elimina el rebote. El número de interrupciones registrado por cada pulsación dependerá del pulsador y de las características del rebote. En la siguiente sección se estudian métodos para reducir las detecciones múltiples.

4. Rebotes en el pulsador

El ejemplo anterior permite observar un problema común al trabajar con botones mecánicos: el rebote. Al pulsar el botón una sola vez, pueden producirse varias transiciones rápidas entre HIGH y LOW antes de que la señal se estabilice.

Este fenómeno se conoce como *rebote* o *bouncing*, como se muestra en la Figura 2. Para evitar detectar un mismo evento varias veces, es necesario implementar un mecanismo de antirrebote, que puede realizarse mediante software, hardware o una combinación de ambos.

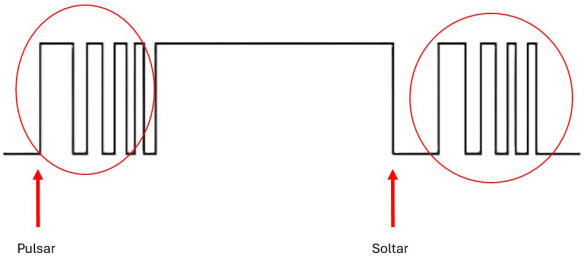


Figura 2: Ejemplo de rebote en un pulsador.

1. Antirrebote por software

Para mitigar el efecto del rebote mediante software, podemos comprobar el tiempo transcurrido entre disparos consecutivos de la interrupción. Si este tiempo es menor que un determinado umbral, simplemente se ignora la interrupción. En otras palabras, definimos una *zona muerta* durante la cual cualquier cambio en el estado del botón es descartado. Este método es sencillo de implementar, aunque introduce un pequeño retardo en la respuesta del sistema. Para programarlo puede ser útil usar la función `millis()` que permite medir el tiempo transcurrido desde que se inició el programa.

2. Antirrebote por hardware

Alternativamente, se puede implementar un filtro hardware que estabilice la señal del pulsador. Un condensador en serie a una resistencia forma un filtro RC, que suaviza los rebotes mecánicos: el condensador se carga y descarga gradualmente, evitando que las transiciones rápidas generadas por el rebote lleguen directamente al microcontrolador.

La figura 3 muestra el esquema de conexión del circuito antirrebote por hardware. La resistencia de 10 kΩ actúa como pull-up pasivo, manteniendo el pin en nivel alto (HIGH) cuando el botón no está presionado. El condensador de 100 nF, conectado entre el pin y GND, forma un filtro RC junto con la resistencia de pull-up. Además, se incorpora una resistencia de 150 Ω en serie con el pulsador para limitar la corriente inicial de descarga del condensador.

La constante de tiempo del filtro determina la rapidez con la que cambia la tensión del condensador. Durante la liberación del pulsador, el condensador se carga a través de la resistencia de pull-up y la resistencia serie. La constante de tiempo se calcula como:

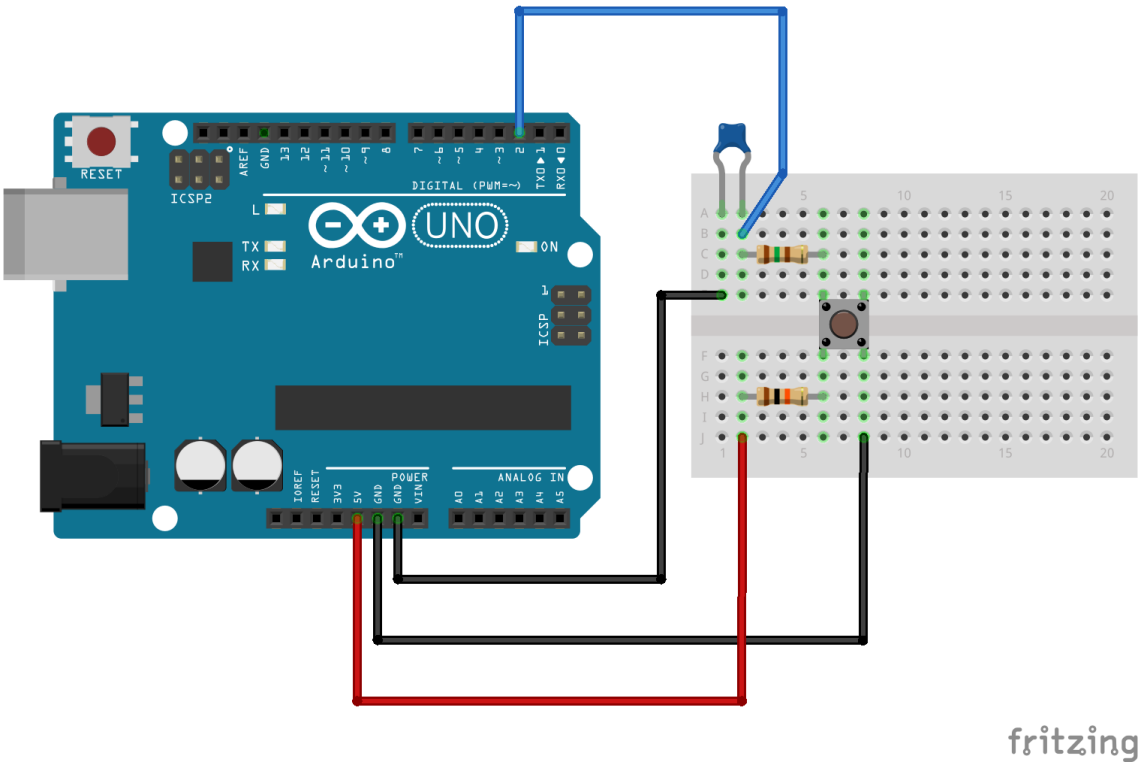


Figura 3: Ejemplo de circuito antirrebote por hardware.

$$\tau_{\text{carga}} = (R_{\text{pull-up}} + R_{\text{serie}}) \cdot C = (10\,000 + 150) \cdot 100 \cdot 10^{-9} = 1{,}015\,\text{ms}$$

Al pulsar, el condensador se descarga a través de la resistencia de $150\,\Omega$ en serie con el pulsador hacia tierra, por lo que la descarga es mucho más rápida:

$$\tau_{\text{descarga}} = R_{\text{serie}} \cdot C = 150 \cdot 100 \cdot 10^{-9} = 15\,\mu\text{s}$$

Este circuito suaviza las variaciones de tensión provocadas por los rebotes mecánicos del pulsador, aunque no garantiza eliminarlos por completo. Además, el filtrado es diferente al pulsar y al soltar el botón, debido a los distintos caminos de carga y descarga del condensador.

Por este motivo, el antirrebote por hardware puede combinarse con el antirrebote por software, que permite ignorar cambios sucesivos durante un intervalo de tiempo determinado.

Ejercicios

Ejercicio 1

Aplica un filtro antirrebotes mediante métodos software sobre el método con interrupciones modificando `boton_interrupcion.ino`. Al fichero resultante llámalo `boton_interrupcion_antirrebotes.ino`. Añade después el filtro RC para un filtrado por hardware y comenta brevemente si cambia algo y por qué.

Ejercicio 2

Rediseña el esquema de conexión, para dar soporte a dos botones que a su vez operan sobre dos LEDs (o un LED RGB) de forma independiente. Así, al pulsar uno de los botones, se encenderá un LED (y se

apagará al soltar dicho botón); y, al pulsar el otro botón, el otro LED cambiará de estado: se encenderá con una pulsación y se apagará al volver a pulsarlo. Para este ejercicio debes utilizar interrupciones y algún método de antirrebotes.

Entrega de la práctica

La entrega se realizará en el repositorio individual de Aula Virtual. Cada repositorio debe contener al menos el código producido así como un archivo de texto que incluya la memoria de la práctica con los siguientes elementos:

La memoria de prácticas debe contener los siguientes elementos:

- Nombre de los estudiantes.
- Objetivo de la práctica.
- Resumen del trabajo realizado.
- Particularidades: Problemas encontrados, funcionalidad extra, etc.
- Vídeo del funcionamiento del ejercicio.
- Esquemático del circuito (Fritzing, Tinkercad, etc.).